Submission Worksheet

Submission Data

Course: IT114-003-F2025
Assignment: IT114 Milestone 2 - Hangman
Student: Alanor S. (ass89)
Status: Submitted | **Worksheet Progress:** 100%
Potential Grade: 10.00/10.00 (100.00%)
Received Grade: 0.00/10.00 (0.00%)
Started: 12/8/2025 10:53:54 AM
Updated: 12/8/2025 10:48:22 PM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT114-003-F2025/it114-milestone-2-hangman/grading/ass89
View Link: https://learn.ethereallab.app/assignment/v3/IT114-003-F2025/it114-milestone-2-hangman/view/ass89

Instructions

1. Refer to Milestone2 of Hangman / Word guess
    1. Complete the features
2. Ensure all code snippets include your ucid, date, and a brief description of what the code does
3. Switch to the `Milestone2` branch
    1. `git checkout Milestone2`
    2. `git pull origin Milestone2`
4. Fill out the below worksheet as you test/demo with 3+ clients in the same session
5. Once finished, click "Submit and Export"
6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone2`
    4. On Github merge the pull request from `Milestone2` to `main`
7. Upload the same PDF to Canvas
8. Sync Local
    1. `git checkout main`
    2. `git pull origin main`

- Complete each section and task sequentially.
- Review the details and validation criteria for each task.
- Ensure subtasks are completed before the parent task.

Section #1: (1 pt.) Payloads

Progress: 100%

☰ Task #1 (1 pt.) - Show Payload classes and subclasses

Progress: 100%

Details:

- Reqs from the document
  - Provided Payload for applicable items that only need client id, message, and type
  - PointsPayload for syncing points of players
  - Each payload will be presented by debug output (i.e, properly override the toString() method like the lesson examples)

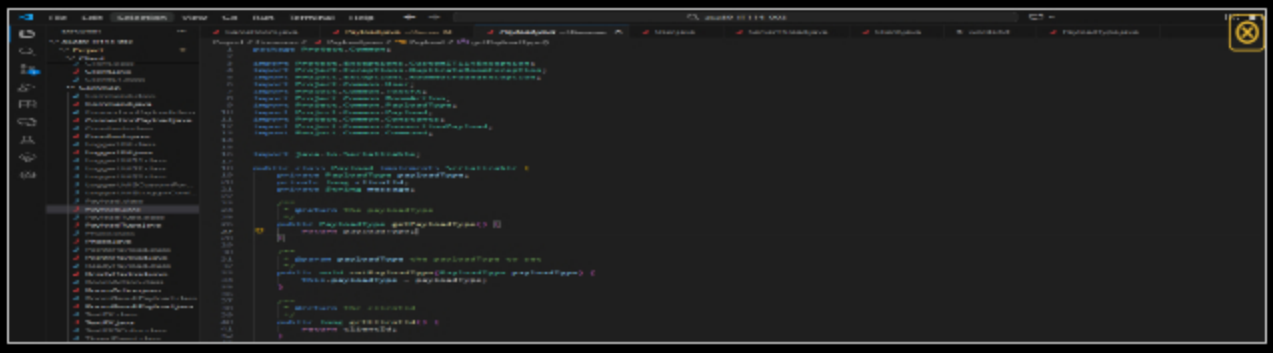
Part 1:

Progress: 100%

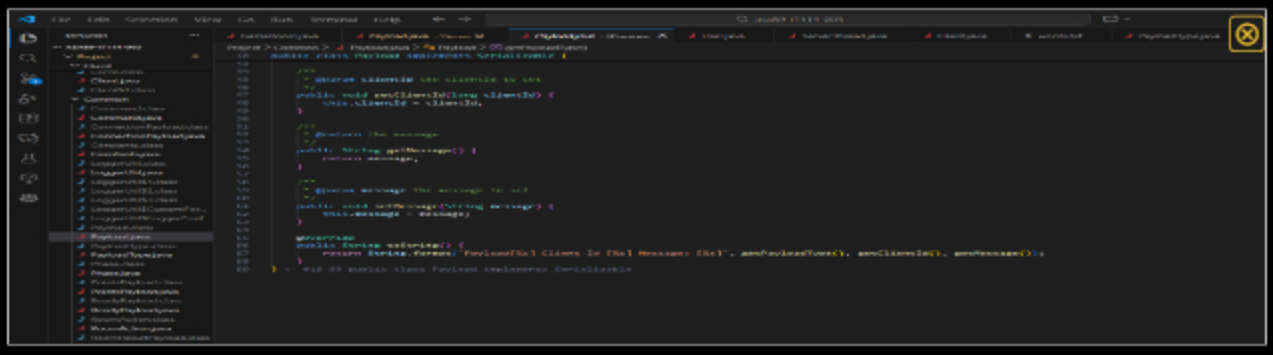
Details:

- Show the code related to your payloads (Payload, PointsPayload, and any new ones added)
- Each payload should have an overriden toString() method showing its internal data

Connection Payload from Common



Payload from Common PT1

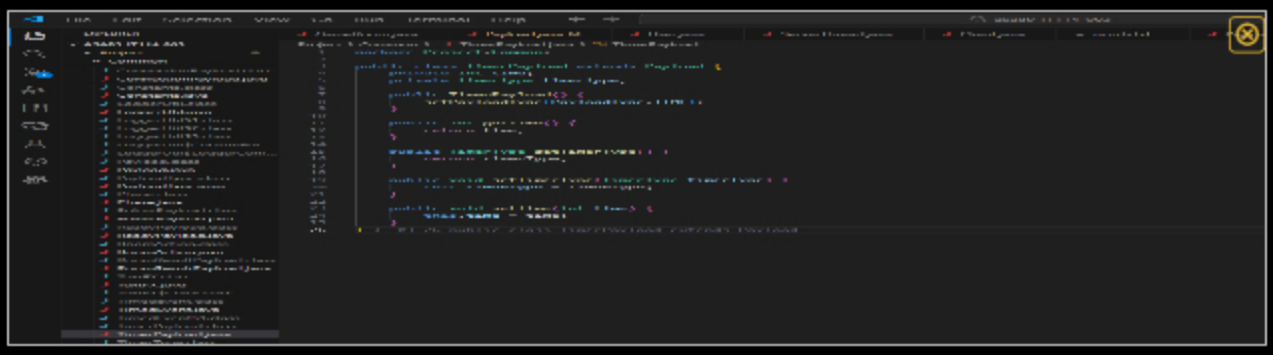


Payload from Common PT2

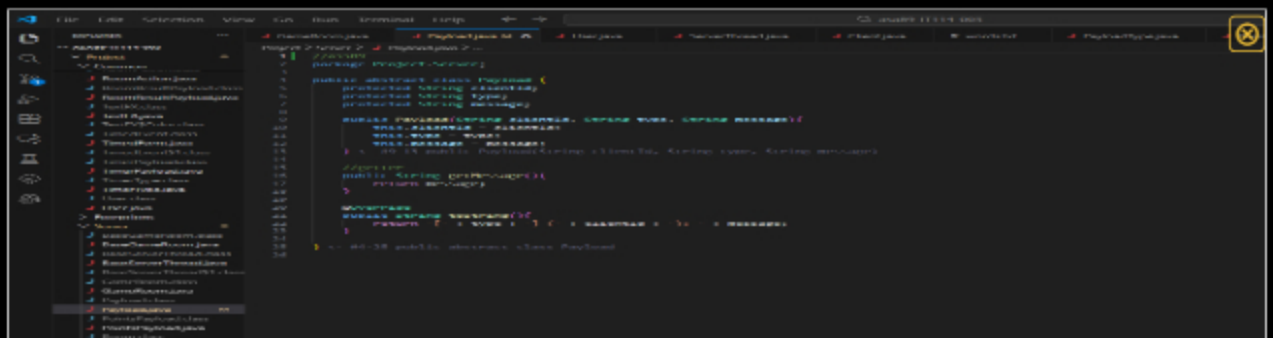
PayloadType from Common

PointsPayload from Common

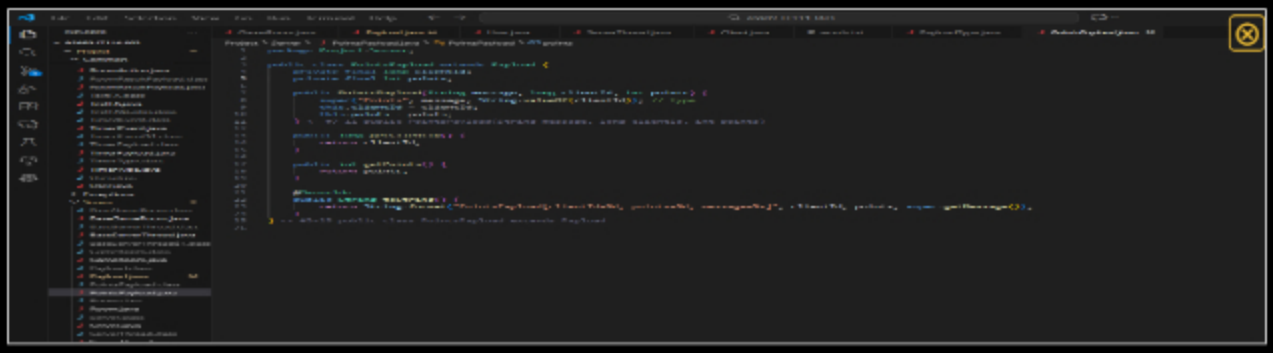
RoomResults Payload from Common



TimerPayload from Common



Payload from Server



PointsPayload from Server

Saved: 12/8/2025 8:07:43 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the purpose of each payload shown in the screenshots and their properties

Your Response:

Payload.java in Common folder:

- base class for most payloads sent between client and server. It has 3 core pieces of data:

1. payloadType: tells the reciever what kind of message this is (like for example MESSAGE, READY, POINTS etc)
2. clientId: identifies which client sent the payload
3. message: a string field, basically a foundation that all other specialized payloads extend

ConnectionPayload in Common:

- it extends Payload and adds one more property--> clientName
- clientName is the username the client provides when connecting. It's used to identofy the client before they recieve an official client ID

PointPayload in Common:

- specialized payload that represents changes in the player's score
- points: the number of points the client earned, which is sent from the server to clients to sync scores during the game

ReadyPayload in Common:

- indicates whether a client is ready to start the game
- isReady: a boolean showing if the user entered the ready command; used by the server to coordinate when all players have signaled readniess and the game can begin

RoomResultPayload in Common:

- represents a list of rooms available on the server
- rooms: a list of room names (List); it's sent to clients when they request the room list

TimerPayload in Common:

- handles countdowns or timed phases in the game
- time: an integer representing the time
- timeType: an enum specifying what timer is running; synchronizes all clients so they get the same timing events

Payload in Server:

- has its own abstract Payload class
- clientId: the ID of the client as a string
- type: the type of the payload (used internally by the server)
- message: the main content of the payload; this server-side version is used for local handling and logging of messages

PointsPayload in Server:

- represents a sompleted scoring event
- clientId: which client has earned points
- points: how many points they recieved; used for logging and internal server communication before it's translated into the Common version of the payload sent to clients

Saved: 12/8/2025 8:07:43 PM

Section #2: (4 pts.) Lifecycle Events

Progress: 100%

Task #1 (0.57 pts.) - GameRoom Client Add/Remove

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the `onClientAdded()` code
- Show the `onClientRemoved()` code

Show the onClientAdded() code Show the onClientRemoved() code

Saved: 12/8/2025 8:22:29 PM

Part 2:

Progress: 100%

Details:

- Briefly note the actions that happen in `onClientAdded()` (app data should at least be synchronized to the joining user)
- Briefly note the actions that happen in `onClientRemoved()` (at least should handle logic for an empty session)

Your Response:

When a new client joins the game room, the onClientAdded() method makes sure that the client's local state is synchronized with the current session. Specifically, it updates the joining client with the current phase of the game, the ready status of all players, and the turn status. This makes sure that the new client sees the correct game state immediately upon joining, without affecting the turn order or changing the other players' data.

When a client leaves the game room, the onClientRemoved() method manages several key tasks to maintain session integrity. It removes the departing layer from the turn order and checks whether the room is now empty. If the room is empty, it resets timers for rounds, turns, and ready status, and calls onSessionEnd() to cleanly end the session. If the player who has left was the current player in the turn order, the method automatically advances to the next player to continue gameplay.

Saved: 12/8/2025 8:22:29 PM

Task #2 (0.57 pts.) - GameRoom Session Start

Progress: 100%

Details:

- Reqs from document
  - GameRoom loads the word list into memory from a text file for later use
  - First word is randomly chosen from in-memory list
  - Blanks are shared with all players
  - First round is triggered
- Reset/set initial state

Part 1:

Progress: 100%

Details:

- Show the snippet of `onSessionStart()`

Show the snippet of onSessionStart()

Saved: 12/8/2025 8:31:13 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up initial session state for your project) and next lifecycle trigger

Your Response:

When a session begins, the game transitions to the IN_PROGRESS phase, signaling that gameplay can start. The turn order is started by collecting all ready players and shuffling them ti create a randomized order of play. The round counter is reset to zero, and the word list for game is loaded from words.txt. After this setup, the session immediately triggers the next lifecycle event by calling onRoundStart(), which begins the first round of the game, and initializes round-specific state (like picking a new word, clearing suggested letters, resetting strikes etc). It makes sure that the game session is fully prepared and transitions smoothly to active gameplay.

Saved: 12/8/2025 8:31:13 PM

Task #3 (0.57 pts.) - GameRoom Round Start

Progress: 100%

Details:

- Reqs from Document
  - GameRoom round timer begins

Part 1:

Progress: 100%

Details:

- Show the snippet of `onRoundStart()`

Show the snippet of onRoundStart()

Saved: 12/8/2025 8:44:18 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up the round for your project)

Your Response:

onRoundStart() prepares both the game state and the players for a new turn-based round. First, it resets the round timer and the turn status for all clients, making sure that there are no leftover state from the previous round carries over. The round counter is incremented, and a new word is selected from the word list while the display blanks are initialized to underscore characters. The previously guessed letters are cleared, and the global strike count is reset to zero, creating a clean slate for the round. The server then notifies all clients of the new round and the current blanked word display, making sure that everyone sees the correct state. OnTurnStart() is then called to determine whcih player begins, starting the turn-based cycle where each player can guess a letter, attempt to guess the full word, or skip their turn.

Saved: 12/8/2025 8:44:18 PM

Task #4 (0.57 pts.) - GameRoom Turn Start

Progress: 100%

Details:

- Reqs from Document
  - Pick next Player (Initially random, then round-robin)
  - Reset turn related status to allow player to do actions
  - GameRoom Turn timer begins

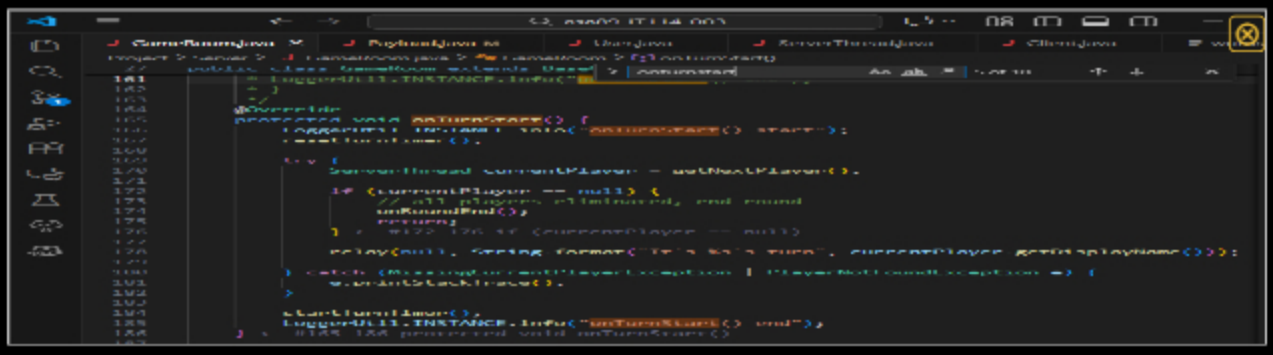
- GameRoom turn timer begins

Part 1:

Progress: 100%

Details:

- Show the snippet of onTurnStart()



Show the snippet of onTurnStart()

Saved: 12/8/2025 9:05:03 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up the turn for your project)

Your Response:

When a round begins, setTurnOrder() collects all ready players and shuffles them to establish a randomized turn order. The currentTurnClientId tracks whose turn it is, and getNextPlayer() advances this ID in round-robin style. At the start of each turn, onTurnStart() resets the turn timer, identifies the current player, and notifies all clients that is that player's turn. Player actions are then processed, after which the player is marked as having taken their turn and other clients are updated via sendTurnStatus(). onTurnEnd() either advances to the next player or ends the round if all players have taken their turn.

Saved: 12/8/2025 9:05:03 PM

Task #5 (0.57 pts.) - GameRoom Turn End

Progress: 100%

Details:

- Reqs from Document

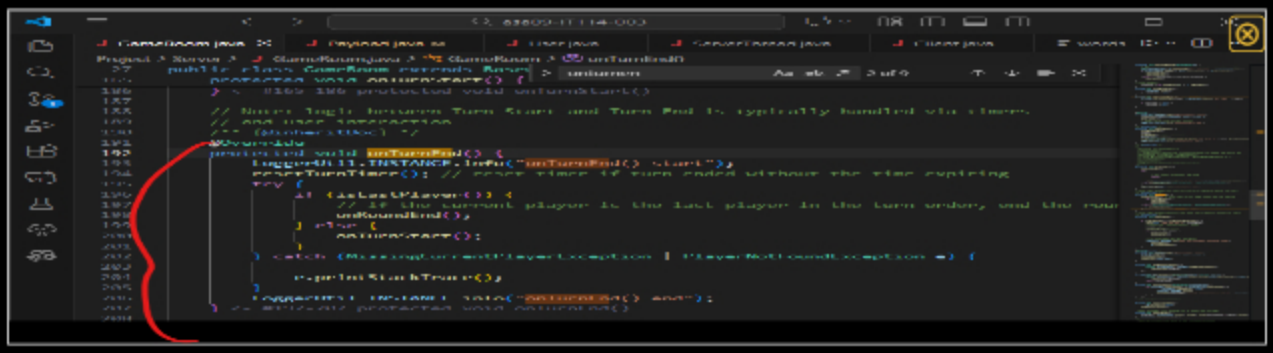
- **Condition 1:** Turn ends when Player guesses a letter or word (and word isn't complete)
- **Condition 2:** Turn ends when the Turn Timer expires
- **Condition 3:** Turn ends when the current Player Skips their turn
- If not the last player, trigger turn start logic
- If the last player, trigger round end logic
- If end condition is met, trigger session end

Part 1:

Progress: 100%

Details:

- Show the snippet of `onTurnEnd()`



Show the snippet of onTurnEnd()

Saved: 12/8/2025 9:25:27 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up the turn for your project)

Your Response:

onTurnEnd() manages the transition between players' turns and makes sure the game flow moves on nicely. When a player's turn ends, either because they completed their action or their turn timer expired, the method first resets the turn timer to prevent any leftover countdowns. It then checks whether the current player is the last in the turn order using isLastPlayer(). If they are, onTurnEnd() triggers the end of the round by calling onRoundEnd(). Otherwise, it moves on to the next player's turn by calling onTurnStart(), which updates the currentTurnClientId and notifies all clients whose turn it is. It accurately tracks each player's actions while keeping the turn-based setup of the game. It also takes care of unusual situations, like missing or removed players, by catching errors and stopping the game from stalling.

Saved: 12/8/2025 9:25:27 PM

Saved: 12/8/2025 9:23:27 PM

Task #6 (0.57 pts.) - GameRoom Round End

Progress: 100%

Details:

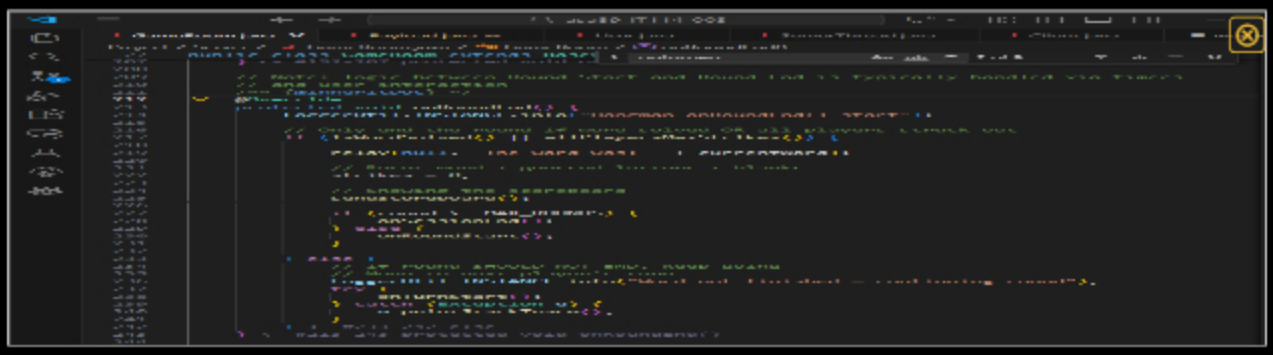
- Reqs from Document
  - **Condition 1:** Round ends when word is completed
  - **Condition 2:** Round ends when Hangman Strikes reach limit
  - **Condition 3:** Round ends when the last player's turn finishes
  - If word was completed, pick new word, reset hangman, sync blanks (if not last round)
  - Send the in-progress scoreboard to all clients sorted by highest points to lowest
  - Trigger Round Start if end condition not met; otherwise, trigger session end

Part 1:

Progress: 100%

Details:

- Show the snippet of `onRoundEnd()`



Show the snippet of onRoundEnd()

Saved: 12/8/2025 9:20:26 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., cleanup, end checks, and next lifecycle events)

Your Response:

onRoundEnd() handles the conclusion of a single round and determines how the game should proceed based on the current state. When called, it first checks whether the round should truly

end, which happens if the word has been fully solved or if all players have reached the maximum number of strikes. If either condition is met, the method reveals the current word to all clients, resets the strike counter, and sends an updated scoreboard showing each player's points. It then checks whether the session has reached maximum nunmber of rounds. If it does, it ends the session by calling onSessionEnd(). Otherwise, it starts a new round with onRoundStart(), which resets turn status, picks a new word, clears guessed letters, and prepares the next round. If the word is still unsolved and players have strikes remaining, the method moves on to the next player's turn by calling onTurnStart().

Saved: 12/8/2025 9:20:26 PM

Task #7 (0.57 pts.) - GameRoom Session End

Progress: 100%

Details:

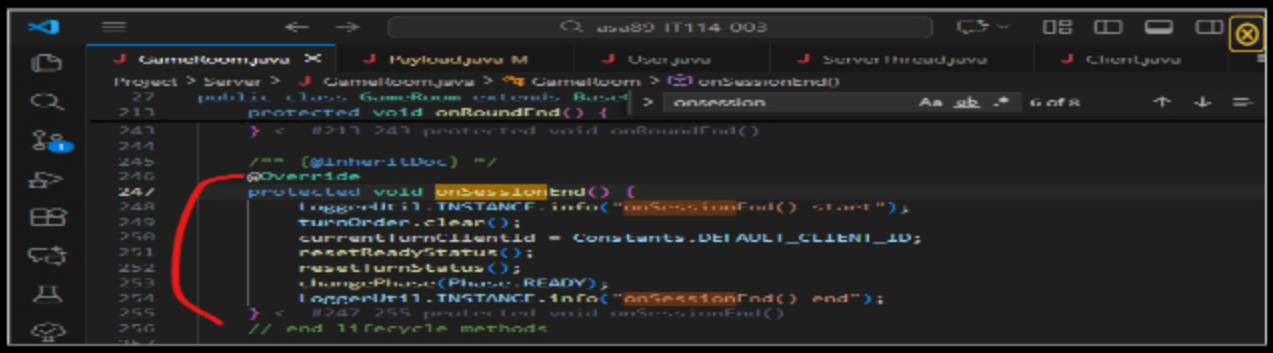
- Reqs from Document
  - **Condition 1:** Session ends when X rounds have passed
  - Send the final scoreboard to all clients sorted by highest points to lowest (include a game over message)
  - Reset the player data for each client server-side and client-side (do not disconnect them or move them to the lobby)
  - A new ready check will be required to start a new session

Part 1:

Progress: 100%

Details:

- Show the snippet of `onSessionEnd()`



Show the snippet of onSessionEnd()

Saved: 12/8/2025 9:29:35 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., cleanup/reset, next lifecycle events)

Your Response:

onSessionEnd() handles the cleanup and reset of the game room once a session concludes, either because the maximum number of rounds has been reached or all players have completed their turns. It clears the turn order, resets the current turn client ID to a default value, and resets both the ready status and turn status of all clients. It also changes the game phase back to READY, meaning that the room is now idle and ready for a new session. Basically; it makes sure that all session-specific data is cleared and it returns the room to a clean, consistent state. This way, new sessions can start fresh, without any leftover state from the previous game. It serves as a key step for finishing the session and getting the room ready for the next lifecycle events.

Saved: 12/8/2025 9:29:35 PM

Section #3: (4 pts.) Gameroom User Action And State

Progress: 100%

Task #1 (1.50 pts.) - Word Logic

Progress: 100%

Details:

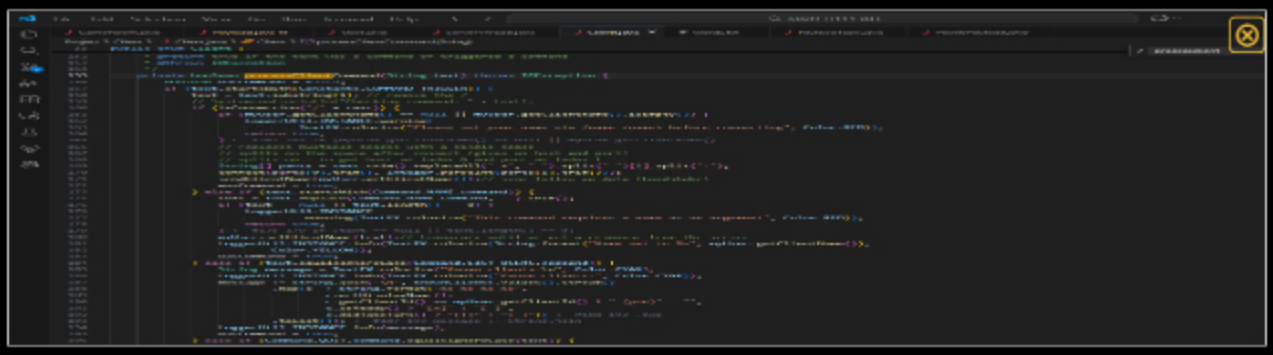
- Reqs from document
  - Command: `/guess <word>`
    - GameRoom will check against the correct word
      - If it matches (exactly)
        - The guesser will receive points for each missing letter and bonus points for solving (i.e., 2x points for each empty space)
          - Points will be stored on the Player/User object
          - Sync the points value of the Player to all Clients
        - A message will be relayed that "X guessed the correct word {word} and got {points} points".
        - The round should end
      - If no matches
        - A strike will be incurred and synced to all Clients
        - A message will be relayed that "X guessed {word} and it was wrong"
        - Their turn should end

Part 1:

Progress: 100%

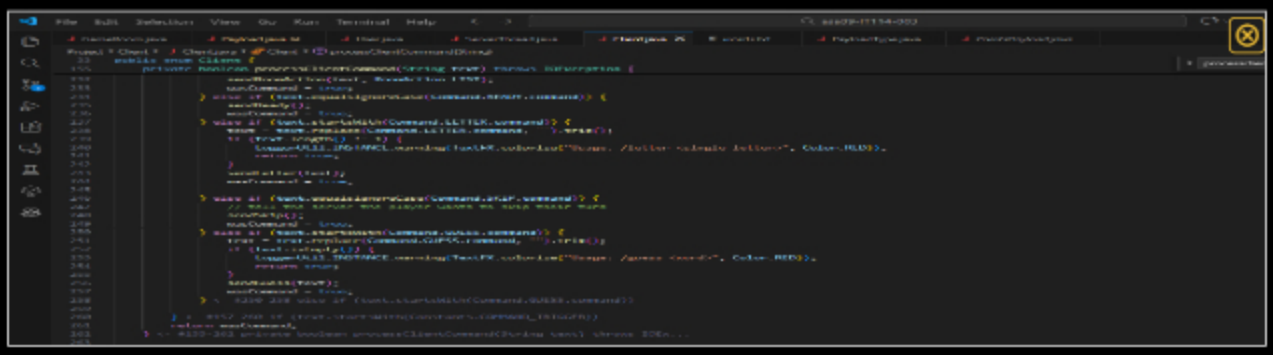
Details:

- Show the code snippets of the following, and clearly caption each screenshot
- Show the Client processing of this command (process client command)
- Show the ServerThread processing of this command (process method)
- Show the GameRoom handling of this command (handle method)
- Show the sending/syncing of the results of this command to users (send/sync method)
- Show the ServerThread receiving this data (send method)
- Show the Client receiving this data (process method)

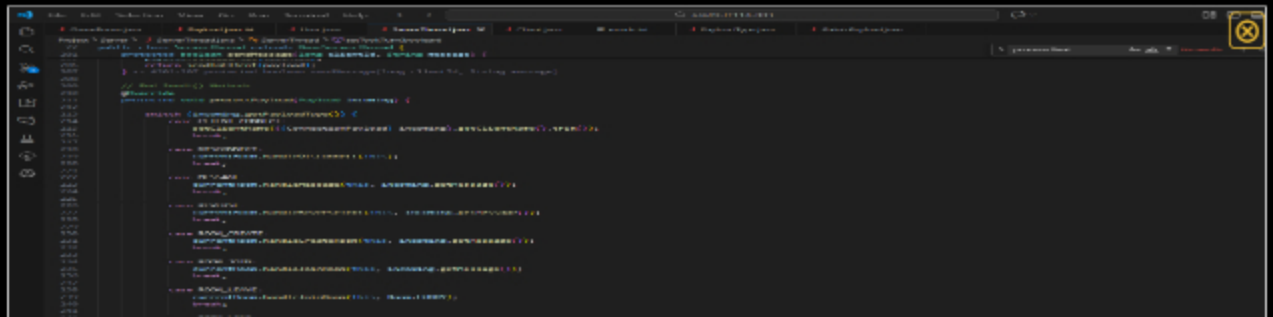


Client side first processClientCommand Pt1

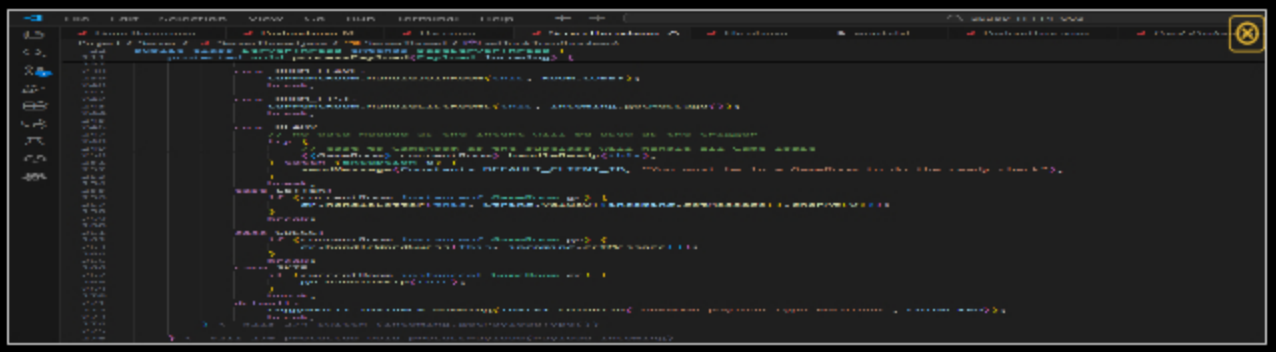
Client side first processClientCommand Pt2



Client side first processClientCommand Pt3 Logic is here

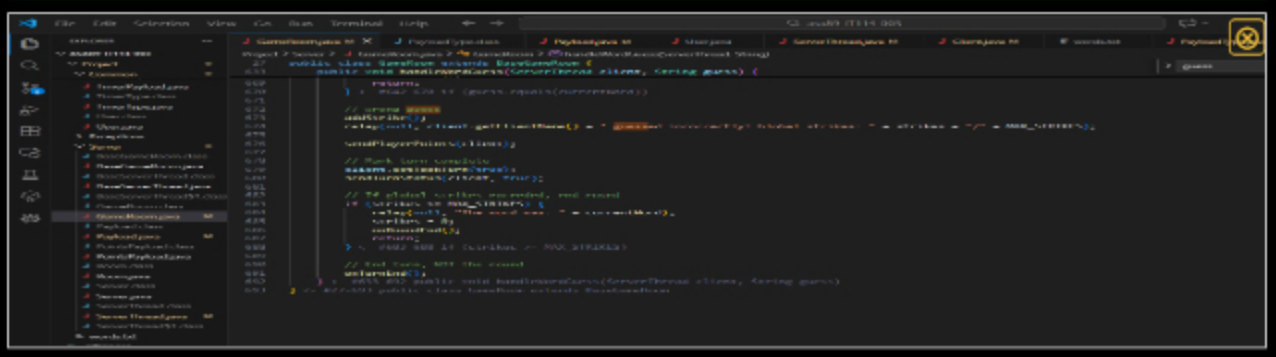


ServerThread next processPayload Pt1



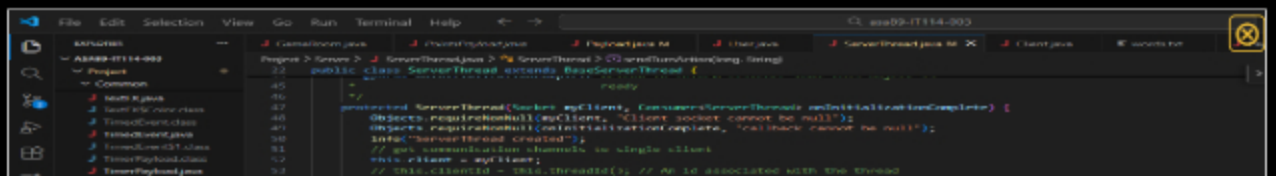
ServerThread next processPayload Pt2 CASE Guess

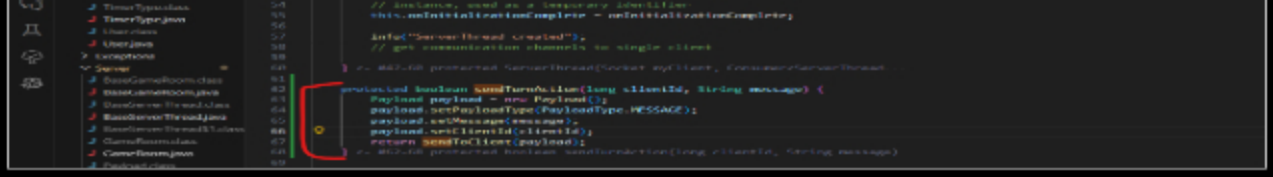
GameRoom handling handleLetter Pt1



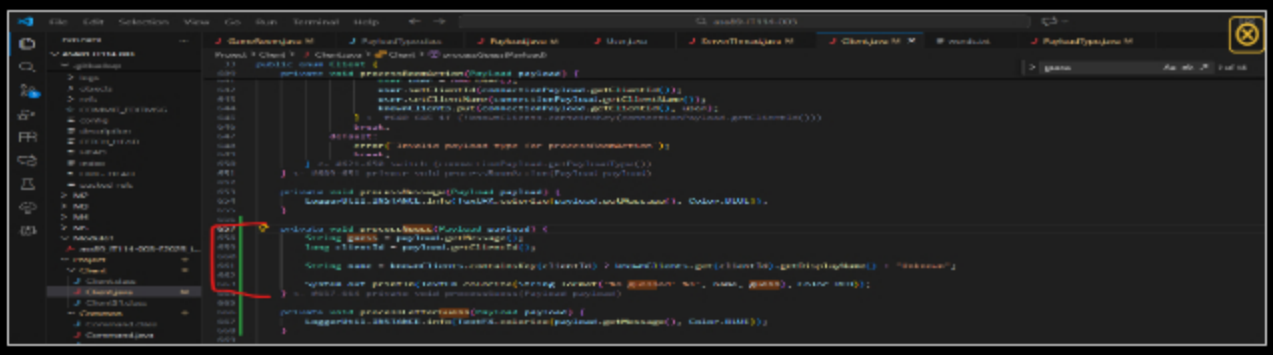
GameRoom handling handleLetter Pt2

Show the sending/syncing of the results of this command to users





Show the ServerThread receiving this data (send method) sendTurnAction



Show the Client receiving this data (process method) processGuess

Saved: 12/8/2025 10:24:39 PM

Part 2:

Progress: 100%

Details:

- Briefly explain/list in order the whole flow of this command being handled from the client-side to the server-side and back

Your Response:

When a player enters the /guess command on the client side, the processClientCommand() method detects the command and calls sendGuess(), which creates a Payload with PayloadType.GUESS and the guessed word, then sends it to the server using sendToServer(). On the server side, the ServerThread recieves this payload in processPayload() and forwards it to the GameRoom via handleWordGuess(). The GameRoom then checks the guess against the correct word, updates the player's points if the guess is correct or increments strikes if it's wrong, and broadcasts the result to all clients using methods like sendMessage() and sendPoints(). Each client recieves the server's payload in listenToServer() and processes it in processPayload(), updating the points, strikes, and displaying a message, completing the round or ending the player's turn as appropriate.

Saved: 12/8/2025 10:24:39 PM

Task #2 (1.50 pts.) - Letter Logic

Progress: 100%

Details:

- Regs from document

Reqs from document

- Command: `/letter <letter>`
  - GameRoom will check the occurrences of the letter that haven't been guessed this round
    - If any matches
      - The guesser will receive points for each slot the letter appears in if it has not been previously guessed for this word
        - I.e., points times the number of matched letters
        - Points will be stored on the Player/User object
        - Sync the points value of the Player to all Clients
      - A message will be relayed "X guessed {letter} and there were {number} {letter}'s which yielded {points} points
      - If word is completed, round should end
      - Otherwise, just their turn should end
    - If no matches
      - A strike will be incurred and synced to all Clients
      - A message will be relayed "X guess {letter} letter, which isn't in the word"
      - If strikes reach the limit the round ends
      - If strikes don't reach the limit the current Player's turn ends

Part 1:

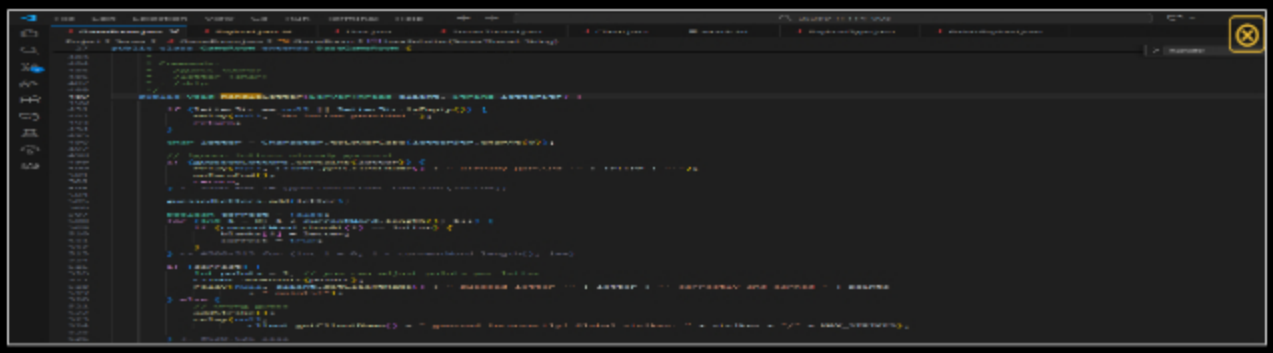
Progress: 100%

Details:

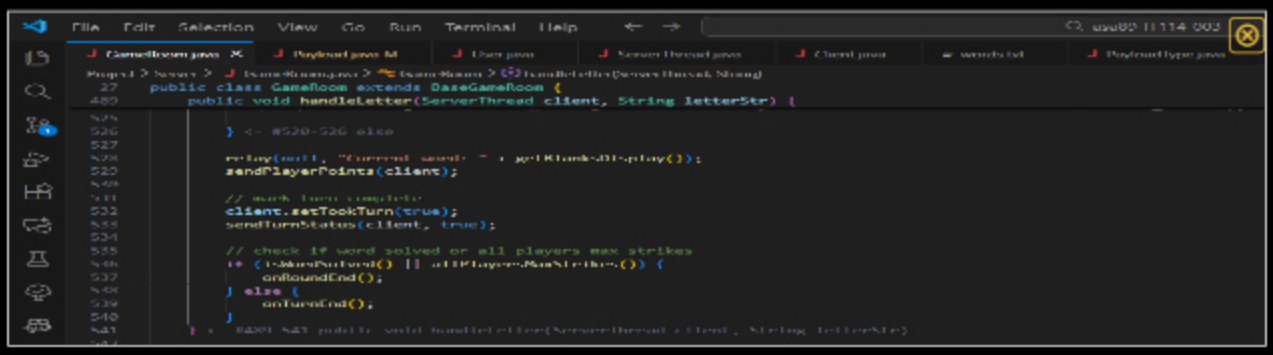
- Show the code snippets of the following, and clearly caption each screenshot
- Show the Client processing of this command (process client command)
- Show the ServerThread processing of this command (process method)
- Show the GameRoom handling of this command (handle method)
- Show the sending/syncing of the results of this command to users (send/sync method)
- Show the ServerThread receiving this data (send method)
- Show the Client receiving this data (process method)

Show the Client processing of this command (process client command)

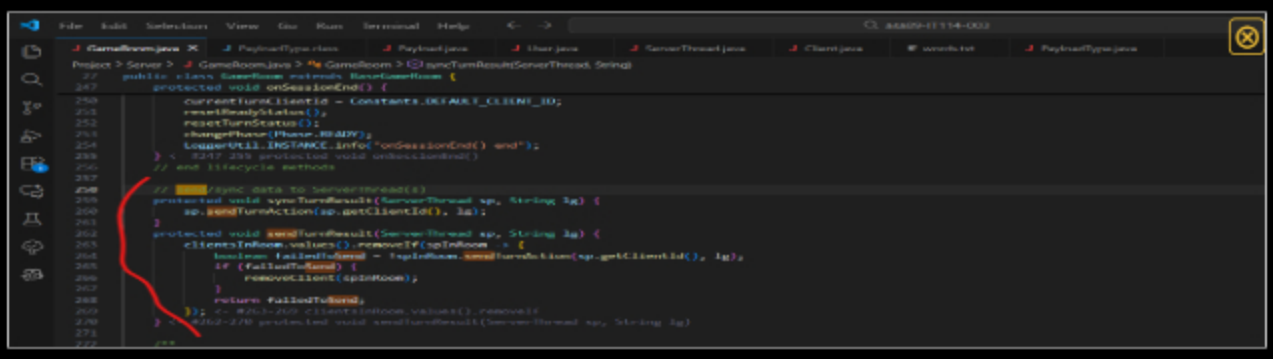
Show the ServerThread processing of this command (process method)



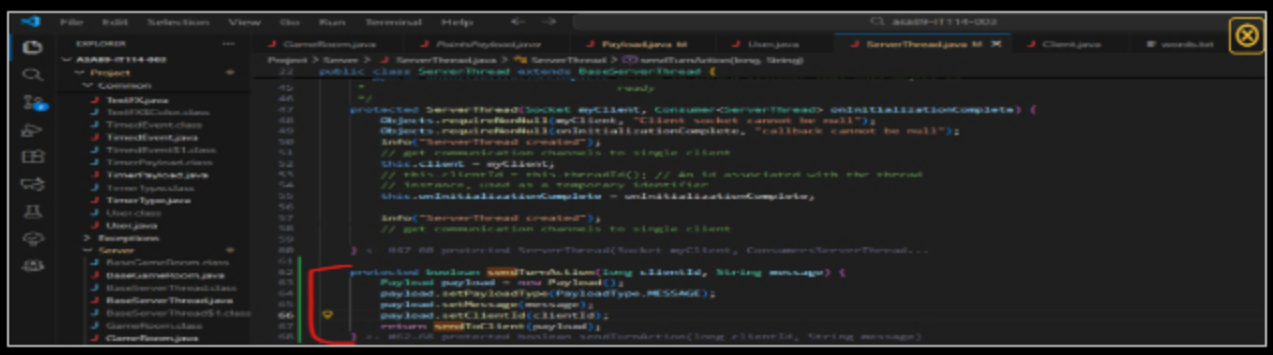
Show the GameRoom handling of this command (handle method) PT1



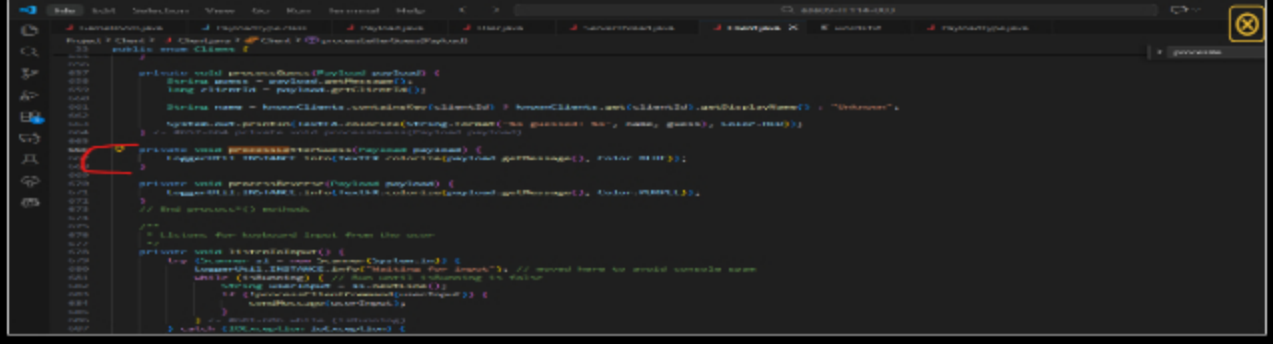
Show the GameRoom handling of this command (handle method) PT2



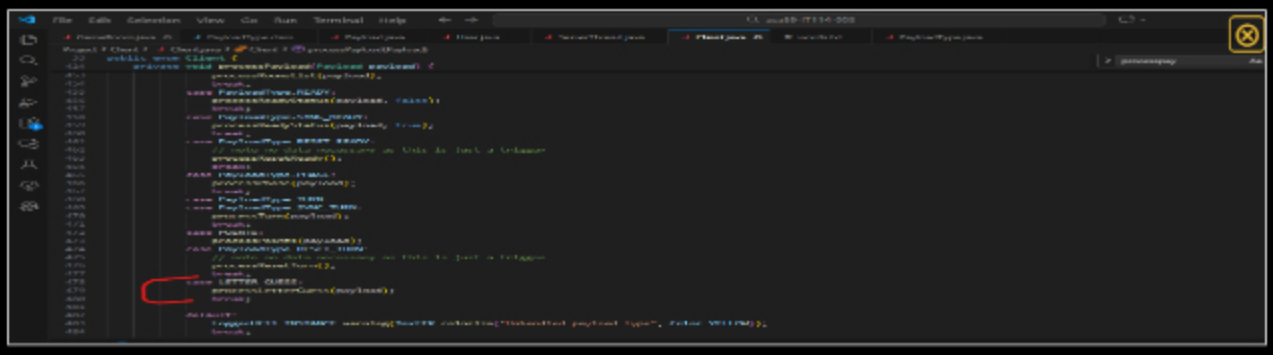
Show the sending/syncing of the results of this command to users (send/sync method) + Show the ServerThread receiving this data



Show the ServerThread receiving this data (send method)



Show the Client receiving this data (process method)



Show the Client receiving this data (process method)

Saved: 12/8/2025 10:41:25 PM

Part 2:

Progress: 100%

Details:

- Briefly explain/list in order the whole flow of this command being handled from the client-side to the server-side and back

Your Response:

When a player types /letter in the client, the Client class detects the command in processClientCommand() and sends a Payload object with type LETTER and the guessed letter in the .message field via sendToServer(). The ServerThread recieves this payload in processPayload() and passes it to the GameRoom by calling handleLetter(), providing tge ServerThread instance and the guessed letter. In GameRoom, the letter is checked against the current word for occurences that haven't been guessed yet. If matches are found, the guesser earns points based on the number of revealed letters that haven't been guessed, the ServerThread's point total is updated, and a Payload is prepared with a message. If the word is completed, the round ends, otherwise the player's turn ends. If there are no matches, the players recieve a global strike, the strike count is synchronized with all clients, and a message is sent to all the clients. The ServerThread uses its sendToClient() methods to distribute these updates to all clients. On the client side, listentToServer() recieves the payloads, and processPayload() handles them, updating the user interface and local player state so each client sees the results of the guess, updates points, and any strikes in real time.

Saved: 12/8/2025 10:41:25 PM

Saved: 12/8/2025 10:41:25 PM

Task #3 (0.50 pts.) - Skip Logic

Progress: 100%

Details:

- Reqs from document
  - Command: `/skip`
    - Skips the player's turn if it's their turn (ends their turn)

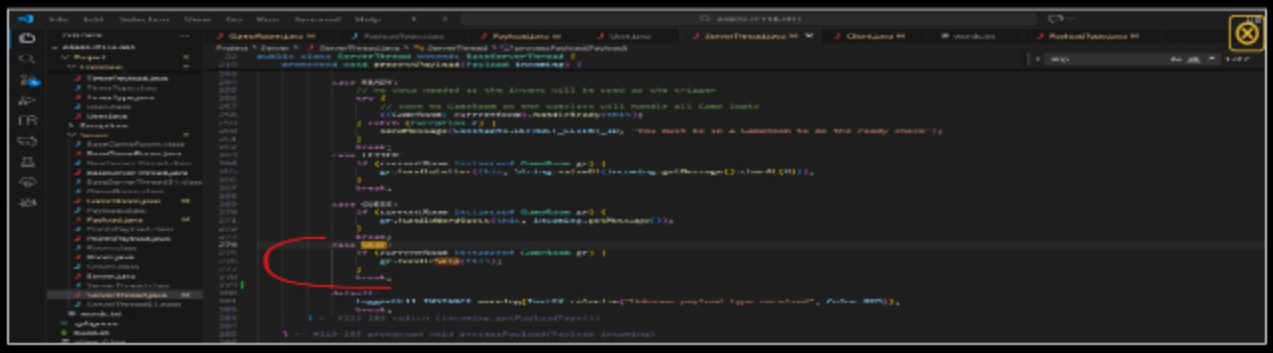
Part 1:

Progress: 100%

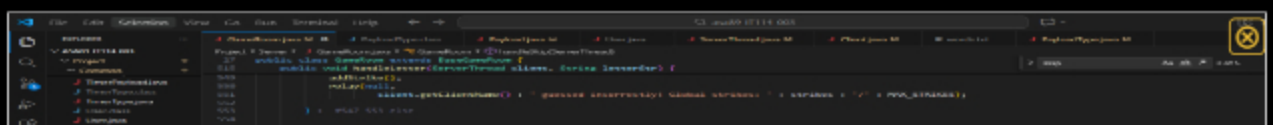
Details:

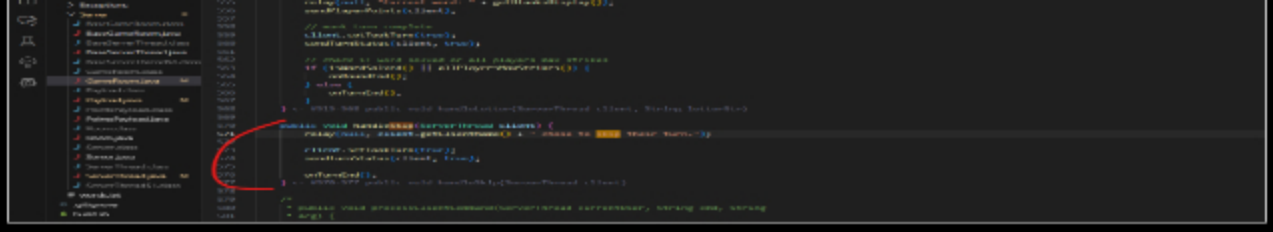
- Show the code snippets of the following, and clearly caption each screenshot
- Show the Client processing of this command (process client command)
- Show the ServerThread processing of this command (process method)
- Show the GameRoom handling of this command (handle method)
- Show the sending/syncing of the results of this command to users (send/sync method)
- Show the ServerThread receiving this data (send method)
- Show the Client receiving this data (process method)

Show the Client processing of this command (process client command)

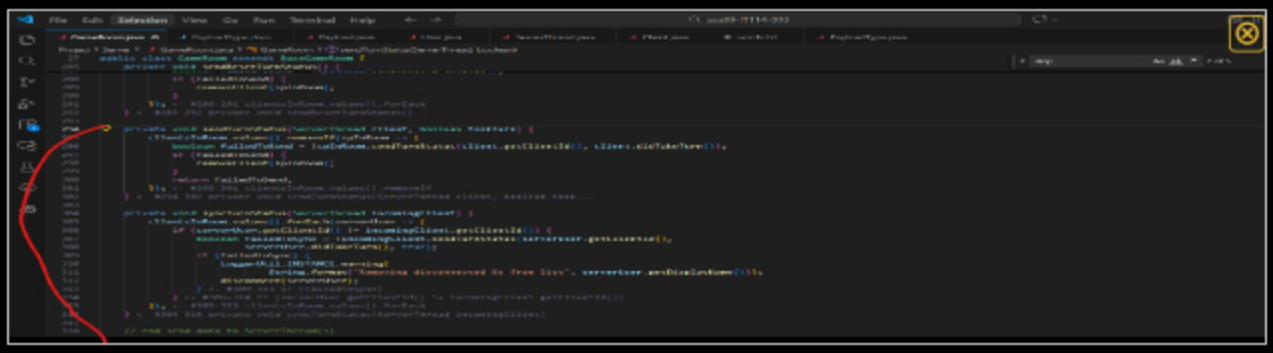


Show the ServerThread processing of this command (process method)

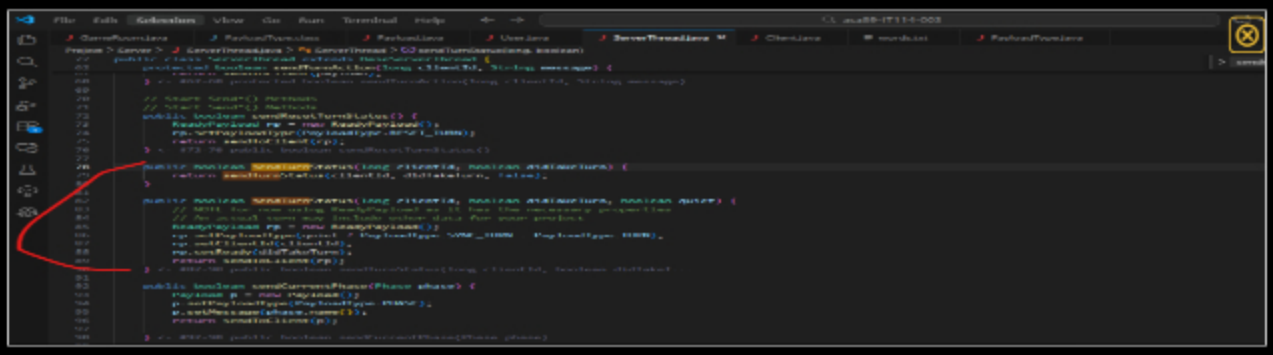




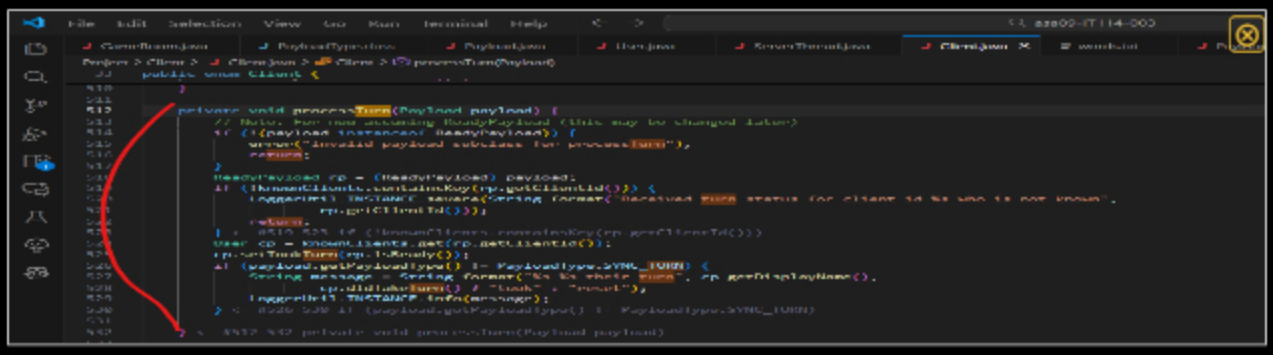
Show the GameRoom handling of this command (handle method)



Show the sending/syncing of the results of this command to users (send/sync method)



Show the ServerThread receiving this data (send method)



Show the Client receiving this data (process method) My skip triggers a turn end

Saved: 12/8/2025 10:47:28 PM

Part 2:

Progress: 100%

Details:

- Briefly explain/list in order the whole flow of this command being handled from the client-side to the server-side and back

Your Response:

When a player types /skip on the client, the processClientCommand() captures the command and calls sendSkip(), which sends a SKIP payload to the server. The ServerThread recieves this payload in processPayload() and calls handleSkip(this) in the GameRoom to mark the player's turn as ended. GameRoom then uses the ServerThread's sendTurnStatus() method to broadcast the turn-ending update to all connected clients. Each client recieves this payload in listenToServer(), and processPayload() calls processTurn(), updating the player's state locally and displaying a message to show the turn has been skipped.

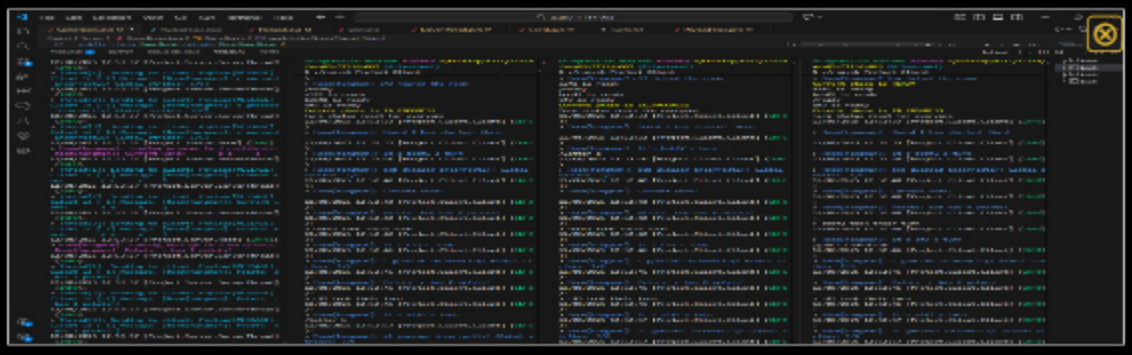
Saved: 12/8/2025 10:47:28 PM

Task #4 (0.50 pts.) - Game Cycle Demo

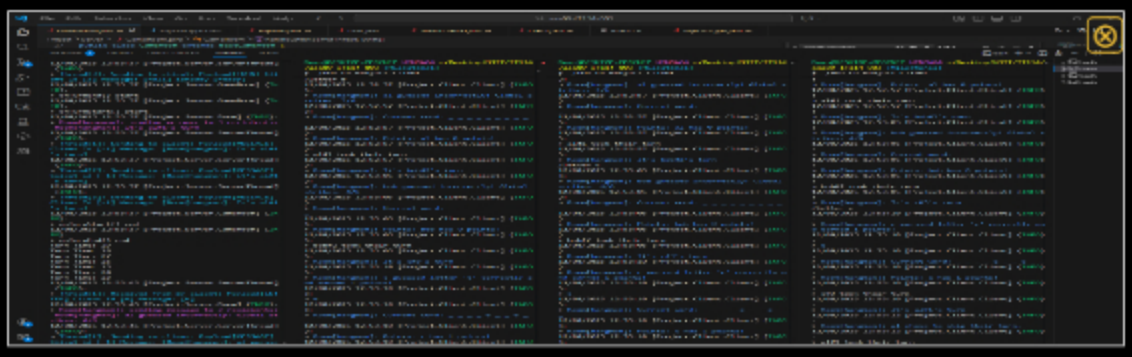
Progress: 100%

Details:

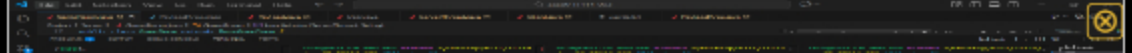
- Show examples from the terminal of a full session demonstrating each command and progress output
- This includes showing blanks, correct letters, incorrect letters, correct words, incorrect words, skips, scores, and scoreboards, etc
- Ensure at least 3 Clients and the Server are shown
- Clearly caption screenshots

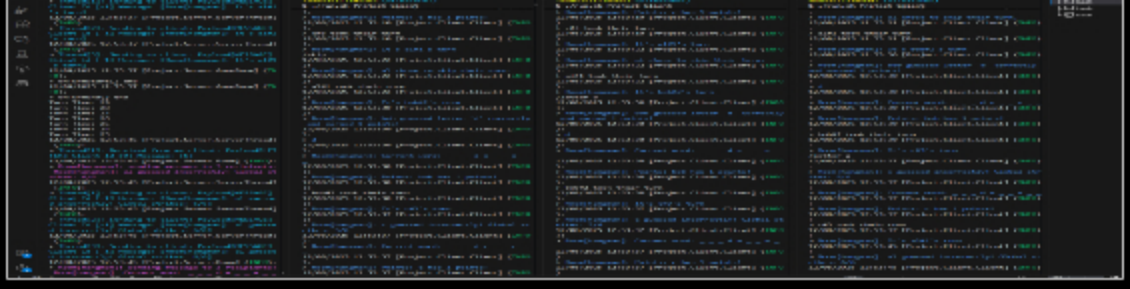


long output Part 1

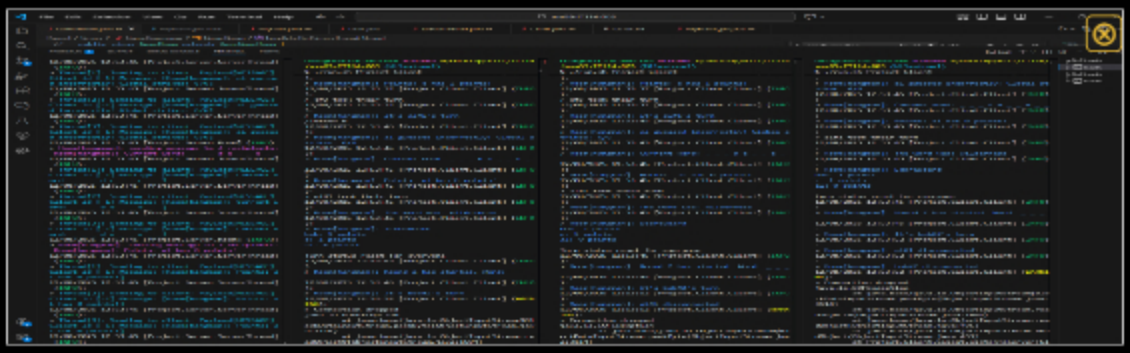


long output Part 2

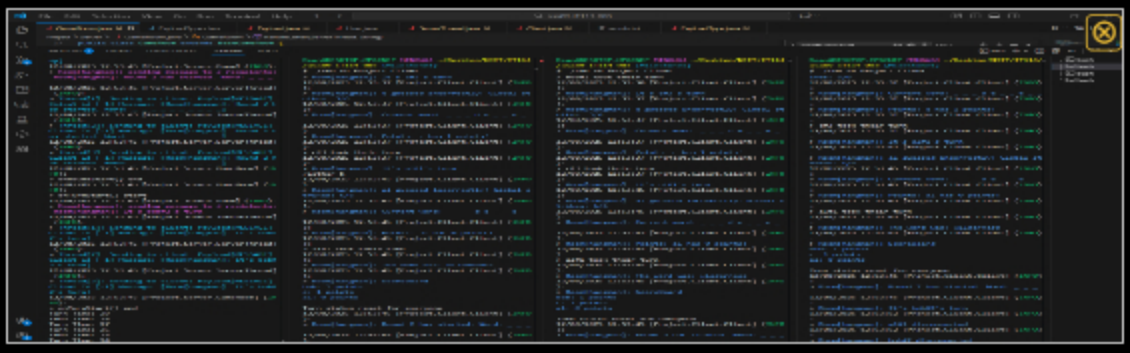




long output Part 3



long output Part 4



long output Part 5

Saved: 12/8/2025 1:00:13 PM

Section #4: (1 pt.) Misc

Progress: 100%

Task #1 (0.33 pts.) - Github Details

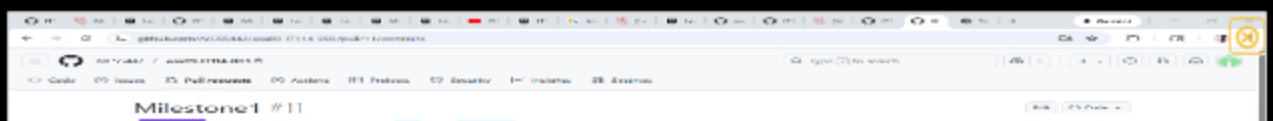
Progress: 100%

Part 1:

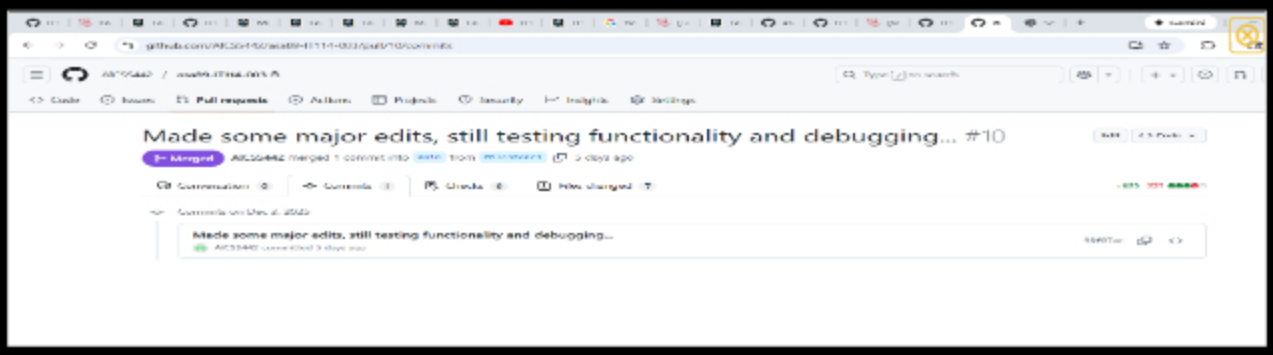
Progress: 100%

Details:

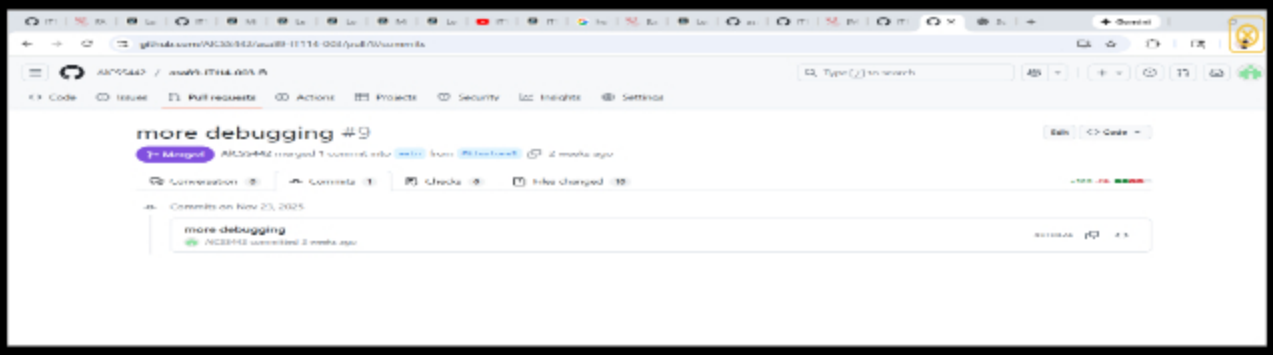
From the Commits tab of the Pull Request screenshot the commit history



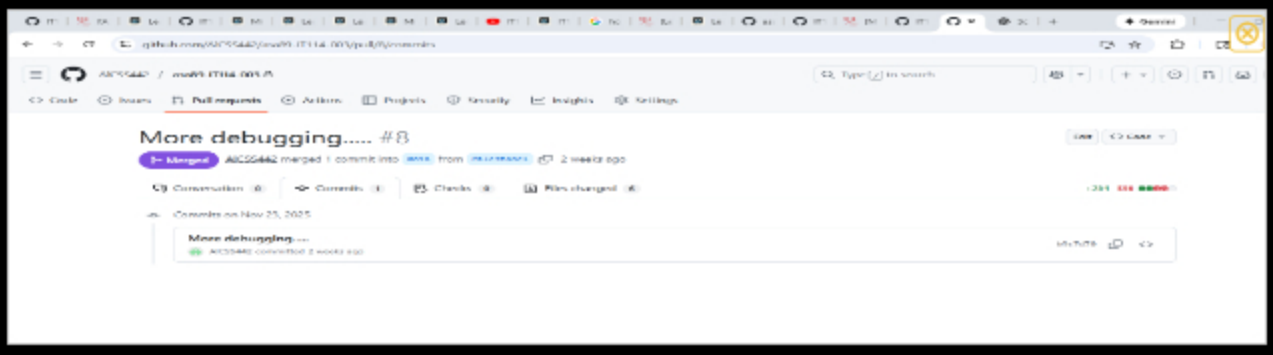
Forgot to create a branch called Milestone2, its under Milestone1



Pt2



PT3



PT4

Saved: 12/8/2025 1:11:00 PM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request (should end in `/pull/#`)

URL #1
https://github.com/AlCSS442/asa89-IT114-003/1

URL
https://github.com/AlCSS442/asa

URL #2
https://github.com/AlCSS442/asa89-IT114-003/0

URL
https://github.com/AlCSS442/asa

URL #3
https://github.com/AlCSS442/asa89-IT114-003/

URL
https://github.com/AlCSS442/asa

URL #4
https://github.com/AlCSS442/asa89-IT114-003/

URL
https://github.com/AlCSS442/asa

Saved: 12/8/2025 1:11:00 PM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

PT1

PT2

Saved: 12/8/2025 1:13:15 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

I learned how important version control is. I was using the wrong version of the template at certain points, which caused a lot of issues that had to be remedied later. Besides tha, I learned that methods that send or sync data to clients don't actually perform the sending themselves. Instead, they build a payload with the data and then call sendToClient(). That sendToClient() method is the place where the actual network communication happens. So on a high level: Each guess is sent as a payload to the server, which processes it and broadcasts updates to all clients. The clients receive these payloads and update their display accordingly, showing the current word, correct guesses, and game messages in real time.

Saved: 12/8/2025 9:41:15 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

I think the easiest part of the project was probably handling server-side events like letter guesses or user disconnections, because you could process the action and log it with LoggerUtil.INSTANCE in one step. For example, after a player guesses a letter, you create the payload, broadcast it to clients, and log the event using something like: LoggerUtil.INSTANCE.info(String.format("Player %s guessed letter: %s", player.getDisplayName(), letter));

Saved: 12/8/2025 10:05:28 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

I think the hardest part was creating and integrating the new payload type for guessed letters and making sure the client properly handled it. It was a little bit of a process because I had to add a new PayloadType, set the guessed letter in the payload on the server, broadcast it, then update the client's processPayload method to handle this new type. I can extend this to creating and integrating new payload types in general as well, that was just a specific example.

Saved: 12/8/2025 10:48:22 PM